

TEKNİK TASARIM BELGESİ

REVİZYON NOTLARI

v1.6 → v1.7

Go Tabanlı Yüksek Performanslı Reverse Proxy
& Yük Dengeleyici

Berk Egemen Oğuz

Hazırlanma Tarihi: 10 Eylül 2026

Sürüm: 1.7

İçindekiler

0. Kapak ve sürüm bilgisi	3
1. Bölüm 3.1 — Klasör ve paket yapısı	3
2. Bölüm 3.3 — Config dosyası şeması	3
3. Bölüm 5.3 — Weighted Round Robin	4
4. Bölüm 5.5 — Backend hata davranışı	5
5. Bölüm 6.2 — Örnek arayüz tasarımı	5
6. Bölüm 7.1, 7.3 ve 11 — Dalların ve pull request akışı	6
7. Bölüm 7.6 — Dağıtım sonrası izleme	7
8. Bölüm 8.1 — Örnek docker-compose.yml	8
9. Bölüm 9 — 12 günlük plan tablosu	8
10. Bölüm 10.3 — Yük testi	8
11. Bölüm 14 — Sonuç ve sonraki adımlar	9
12. Ekleme önerisi: yeni bir bölüm	9

Bu not, 12 günlük uygulama tamamlandıktan sonra belgeyi gerçekleştiren sisteme göre güncellemek için hazırlandı. Her madde, belgedeki **mevcut metni** ve yerine geçecek **yeni metni** verir; Word dosyasındaki şekiller, tablolar ve biçimlendirme korunacak şekilde yalnızca ilgili paragrafları değiştirmen yeterlidir.

Gerekçelerin ayrıntısı depodaki `docs/design-deviations.md` dosyasında; her maddede karşılığı belirtildi.

0. Kapak ve sürüm bilgisi

Mevcut: Sürüm: 1.1 (kapak sayfası; içerik v1.6 olarak adlandırılmış)

Yeni: Sürüm: 1.7 ve tarihi güncelle.

Kapağa şu cümleyi eklemenı öneririm:

Bu sürüm, 12 günlük uygulama planı tamamlandıktan sonra gerçekleştiren sisteme göre revize edilmiştir. Tasarım kararlarının uygulamada nasıl karşılık bulduğu ve hangi noktalarda değiştiği belirtilmiştir.

1. Bölüm 3.1 — Klasör ve paket yapısı

Mevcut: `internal/` altında `config`, `balancer`, `health`, `proxy`, `server`, `observability` listeleniyor.

Yeni: Listeye iki paket ekle:

```
| └─ app/           # modülleri birbirine bağlayan kurulum (wiring)
| └─ integration/   # uçtan uca testler
```

Gerekçe: Wiring `main.go` içinde kalsaydı entegrasyon testleri binary'nin kullandığı zinciri değil kendi kopyalarını doğrulardı. `cmd/lb/main.go` artık yalnızca config okuyup logger kurup `app.New` çağırıyor.

2. Bölüm 3.3 — Config dosyası şeması

Şemaya üç alan eklendi. Örnek YAML bloğunu şu şekilde güncelle:

`listen_addr` **satırının hemen altına:**

```
metrics_addr: ":8081"      # /metrics ve /status
enable_pprof: false       # /debug/pprof uçlarını metrics portuna ekler
```

`timeouts` **bloğunda** `connect_timeout` **satırının altına:**

```
response_timeout: 10s     # backend yanıtı başlamazsa isteği bırak ve tekrar dene
```

Bölümün açıklama maddelerine şunları ekle:

- **metrics_addr** — Gözlemlenebilirlik uçları trafik portundan ayrı bir sokette sunulur. Aynı portta sunulsaydı `/metrics` ve `/status` yolları backend'lere hiç iletilmez ve iç durum load balancer'a erişebilen herkese açılırdı. Bu sunucuya bağlantı limiti uygulanmaz: gözlemlenebilirlik tam da trafik portu dolduğunda çalışmalıdır.
- **enable_pprof** — Go profiling uçlarını açar; heap ve goroutine durumunu dışarı verdiği için varsayılan olarak kapalıdır.
- **response_timeout** — Bağlantıyı kabul edip yanıtı başlamayan bir backend'i sınırlar. Belirtilmezse `read_timeout` değerini alır. Bu alan olmadan, askıda kalan bir backend isteği istemci tarafı `write_timeout` onu kesene kadar tutar ve hata stratejisi hiç devreye giremez; Bölüm 10.4'teki "yavaş yanıt" senaryosunun gereği budur.

Karşılığı: sapma 5 ve 6.

3. Bölüm 5.3 — Weighted Round Robin

Mevcut:

Her backend'e config'te bir ağırlık atanır; seçim bu ağırlıklara orantılı olasılıksal biçimde yapılır.

Yeni:

Her backend'e config'te bir ağırlık atanır; seçim, smooth weighted round robin algoritmasıyla deterministik biçimde yapılır: her turda her backend'in kredisi ağırlığı kadar artar, en yüksek krediye sahip backend isteği alır ve kredisi toplam ağırlık kadar düşer.

Ağırlığa orantılı rastgele seçime kıyasla bu yaklaşım, yapılandırılan oranı yaklaşık değil tam tutturur ve ağır backend'in sırasını döngü boyunca yayar; 5, 1, 1 ağırlıklarında üretilen sıra `a a b a c a a` biçimindedir. Rastgelelik kaynağı gerektirmediği için testler de istatistiksel değil kesin sonuçludur: 1, 2, 3 ağırlıklarıyla 1200 istek tam olarak 200, 400 ve 600 dağılır.

Karşılığı: sapma 2.

4. Bölüm 5.5 — Backend hata davranışı

"Hata Senaryolarına Göre HTTP Yanıtları" listesinin sonuna yeni bir madde ekle:

• **Health check tüm backend'leri sağlıklı işaretlediyse** — İstek yine de havuza sunulur ve denenmemiş backend'lere yönlendirilir; bu düşüş ayrı bir sayaçla (`no_healthy_backend`) kaydedilir. Yük altında sağlık probe'ları istemci trafiğinden önce zaman aşımına uğrar ve tüm backend'ler aynı anda sağlıklı işaretlenebilir; bu durumda tüm trafiği reddetmek yavaş bir sistemi bozuk bir sisteme çevirir. Hâlâ cevap verebilecek bir backend bir denemeye değer, gerçekten ölü bir backend ise tek bir başarısız deneme maliyetiyle istemciye zaten döneceği 503'ü döndürür.

Bunun gözlemlenebilir sonucu şudur: tüm backend'ler sağlıklı işaretliyken ama hâlâ cevap veriyorken, istemci load balancer'ın ürettiği 503 yerine backend'in kendi yanıtını alır.

Gerekçe: Yük testinde bu durum somut olarak yaşandı: doyumluk anında 275.769 istek reddedildi. Karşılığı sapma 4.

5. Bölüm 6.2 — Örnek arayüz tasarımı

Mevcut:

```
type Backend struct {
    Addr    string
    Weight  int
    Healthy bool
}

// internal/health paketi
```

```
type Checker interface {
    Start(ctx context.Context, backends []*Backend)
    IsHealthy(addr string) bool
}
```

Yeni:

```
// internal/balancer paketi
type Backend struct {
    Addr    string
    Weight  int

    // active, bu backend'in o anda işlediği istek sayısıdır; least connections
    // buna göre seçim yapar. Atomik olarak güncellenir.
    active atomic.Int64
}

// internal/health paketi
type Checker interface {
    Start(ctx context.Context, backends []*Backend)
    IsHealthy(addr string) bool
    ReportSuccess(addr string)
    ReportFailure(addr string)
    Reload(ctx context.Context, cfg config.HealthCheck, backends []*Backend)
}
```

Aşağıdaki açıklamayı ekle:

Sağlık durumu `Backend` üzerinde bir alan değil, `Checker` içinde adrese göre tutulur; böylece aktif ve pasif kontrol aynı sayaçları besler ve havuz yeniden kurulduğunda durum korunur. `ReportSuccess` ve `ReportFailure`, Bölüm 6.1'de tanımlanan pasif health check için gereklidir: proxy ilettiği her isteğin sonucunu bildirir, böylece gerçek trafikte hata veren bir backend bir sonraki probe'u beklemeden havuzdan çıkar. `Reload`, config yeniden yüklendiğinde backend kümesini değiştirir.

Karşılığı: sapma 3.

6. Bölüm 7.1, 7.3 ve 11 — Dallanma ve pull request akışı

Bu, belgedeki en büyük fark. Bölüm 7.1'deki dallanma stratejisini ve 7.3'ün ilk maddesini şu metinle değiştir:

Dallanma stratejisi. Proje tek geliştirici tarafından yürütüldüğü için tüm çalışma doğrudan `main` dalına commit edilir; feature dalları ve pull request akışı kullanılmaz. İnceleyecek ikinci bir kişi olmadığında pull request, kalite kapısı işlevi görmeden yalnızca süreç yükü yaratır.

Kalite kapısı bunun yerine CI'dır: `main` 'e yapılan her push'ta `gofmt`, `go vet`, `golangci-lint`, `govulncheck`, derleme ve `-race` altında tüm test paketi çalışır. Ekip büyürse belgede tanımlanan korumalı dal ve pull request akışı devreye alınabilir; CI pipeline'ı bir pull request'in ihtiyaç duyacağı kontrolleri zaten koşturmaktadır.

`experiment/epoll-loop` (Bölüm 4, Faz 2) ve `release/vX.Y.Z` etiketleme yaklaşımı geçerliliğini korur.

Bölüm 11'deki "Versiyon Kontrolü" kriterini şu şekilde güncelle:

- **Versiyon Kontrolü:** Tüm değişiklikler CI'dan geçmiş; v1.0.0 etiketi GitHub Releases'te yayınlanmış ve ilgili Docker imajı container registry'ye gönderilmiş.

Karşılığı: sapma 1.

7. Bölüm 7.6 — Dağıtım sonrası izleme

Mevcut: Grafana servisi `mem_limit: 256m`.

Yeni: `mem_limit: 512m` ve altına not:

Not: Grafana 13, boştaiken 256 MB içinde kalır ancak bir dashboard render edilirken bu limiti aşar ve konteyner OOM ile sonlandırılır (exit 137). Yük altında ölçülen kullanım 331 MiB'dir, bu nedenle limiti 512 MB'dir. Prometheus, on backend scrape edilirken 143 MiB kullanır ve 256 MB limitiyle kalır.

Ayrıca iki küçük düzeltme:

- Prometheus volume yolu belgede `./deploy/prometheus.yml:/etc/prometheus/prometheus.yml` yazıyor; `compose` dosyası `deploy/` içinde olduğu için doğrusu `./prometheus.yml:/etc/prometheus/prometheus.yml`.
- Scrape hedefi olarak yalnızca `compose` servisi listelenmelidir. Hem `loadbalancer:8081` hem `host.docker.internal:8081` aynı anda scrape edilirse, `compose` servisi `metrics` portunu `host`'a yayınladığı için ikisi de aynı sürece ulaşır ve tüm toplama sorguları iki katını gösterir.

Karşılığı: sapma 7.

8. Bölüm 8.1 — Örnek docker-compose.yml

Örnek dosyaya load balancer servisini ekle:

```
loadbalancer:
  build:
    context: ..
    args:
      VERSION: dev
  ports: ["8080:8080", "8081:8081"]
  volumes: ["../configs/lb.example.yml:/etc/lb/config.yml:ro"]
  depends_on: [backend-1, backend-2]
  mem_limit: 128m
```

Not ekle:

İmaj distroless tabanlı olduğu ve kabuk içermediği için konteyner healthcheck'i tanımlanmaz; bu ihtiyacı metrics portundaki `/status` ucu dışarıdan karşılar.

9. Bölüm 9 — 12 günlük plan tablosu

5. gün satırı, mevcut: "Aktif bağlantı sayacı, ağırlıklı olasılıksal seçim, birim testler."

Yeni: "Aktif bağlantı sayacı, smooth weighted round robin ile deterministik ağırlıklı seçim, birim testler."

11. gün satırı: Dokümantasyon çıktısına ek olarak "config hot-reload (SIGHUP)" zaten yazıyor; gerçekleşen kapsamı yansıtmaması için çıktı sütununu "Dayanıklılığı doğrulanmış, SIGHUP ile config yeniden yüklenebilen, dokümanite sistem" olarak güncelleyebilirsiniz.

10. Bölüm 10.3 — Yük testi

Mevcut: "wrk veya ab (Apache Bench) ile artan eşzamanlılık seviyelerinde (örn. 100 → 1.000 → 10.000 bağlantı) throughput ve p50/p95/p99 latency ölçülür."

Yeni:

`ab` (Apache Bench) ile referans ölçümleri alınır; ancak `ab` tek thread'li olduğu için binin üzerindeki eşzamanlılıkta `apr_socket_recv: Operation timed out` hatasıyla ölçüm yapamaz. Daha yüksek seviyeler için bağlantı başına bir goroutine kullanan, keep-alive ile çalışan ve gecikme yüzdelerini kaydeden küçük bir ölçüm aracı kullanılır. Ölçümlerin load balancer'ı mı yoksa yük üreticisini mi tarif ettiğini belirlemek için önce backend'lere doğrudan yük uygulanır.

Hedef maddelerini ölçülen değerlerle değiştir:

- Ölçülen sonuç (10 çekirdekli tek makine; yük üreticisi ve on backend aynı makinede): 100 bağlantıda 40.616 istek/sn, p50 2,1 ms, p95 5,3 ms, p99 7,7 ms; 1.000 bağlantıda 41.208 istek/sn, p95 46,7 ms; 2.000 bağlantıda 41.421 istek/sn, p95 87,5 ms. Hiçbir seviyede istemciye yansıyan hata oluşmamıştır.
- p95 hedefi, beklenen gerçek yük seviyesi belirtilerek ifade edilmelidir. 100 eşzamanlı bağlantıda hedef karşılanmaktadır; 1.000 bağlantıdaki ölçüm, load balancer'ın makineyi yük üreticisi ve backend'lerle paylaştığı koşullarda alınmıştır ve bir dağıtım senaryosunu temsil etmez.

Karşılığı: sapma 8 ve 9.

11. Bölüm 14 — Sonuç ve sonraki adımlar

Giriş paragrafını gerçekleşen duruma göre güncelle:

Bu belgede tanımlanan plan 12 gün içinde tamamlanmış, sistem v1.0.0 olarak etiketlenmiş ve production'a dağıtılabılır bir container imajı üretilmiştir. Üç yük dengeleme algoritması, aktif ve pasif health check, yapılandırılabilir hata stratejileri, rate limiting ve kaynak limitleri, structured logging ile Prometheus tabanlı gözlemlenebilirlik ve SIGHUP ile config yeniden yükleme uygulanmıştır. Kritik paketlerde birim test kapsamı %89–100 aralığında olup, gerçek soketler üzerinden çalışan 22 entegrasyon testi mevcuttur.

12. Ekleme önerisi: yeni bir bölüm

Belgeye "**15. Uygulama Sonrası Değerlendirme**" başlıklı kısa bir bölüm eklemenizi öneririm. İçeriği depodaki üç belgeden derlenebilir:

- Yük testi sonuçları ve bulunan üç darboğaz (`docs/performance-report.md`)
- Tasarımdan sapmalar ve gerekçeleri (`docs/design-deviations.md`)
- Prod hazırlık kriterlerinin karşılanma durumu (`docs/deployment-checklist.md`)

Bu bölüm, belgeyi bir plan dokümanı olmaktan çıkarıp planın nasıl sonuçlandığını da anlatan bir tasarım referansına dönüştürür — Bölüm 1'de belirtilen amacın tam karşılığı.